RL Multimodal Instructions

Working copy of the project brief with the former two-task submission limit removed.

Overview

In this project, you will create multimodal bug-fix conversations using ChatGPT Astra (thinking: low or medium) and Gemini 3.8 Flash.

For this project, we only require **text + image** modality. Example: showing a webapp screenshot and asking to fix the UI.

Each conversation can be a single prompt or multiple turns and must begin with a public GitHub repo link and a specific commit hash. You can use an earlier commit if needed. Include relevant visual context, such as screenshots of the bug, alongside the code context. Give both models the same starting prompt and evidence.

Design challenging tasks where Astra demonstrates a significant, verifiable advantage over Gemini 3.8 Flash. Save both conversations and supporting artifacts in a linked file. In Author Notes, explain why Astra's response is significantly better, citing concrete differences in diagnosis, fix correctness, visual understanding, or validation.

A different person must review each comparison, independently validate whether Astra is indeed significantly better, and document their findings in Reviewer Notes, including any disagreement.

Preference Evaluation: 10 UI Dimensions

Evaluate both Astra and Gemini 3.8 Flash against the UI task prompt, supplied screenshots, and intended behavior. Read each complete transcript and inspect its resulting UI and supporting artifacts. Use the same starting repository commit, prompt, visual evidence, and evaluation conditions for both; record any differences in later assistance that affect comparability.

For each dimension below, compare the two models and select exactly one preference in Labelbox. Use the UI task prompt and supplied evidence as the standard. Apply the criterion only to relevant UI elements and states; do not require unrelated features or redesigns.

Astra better
Astra performs better on this dimension, supported by observable evidence.

Gemini 3.8 Flash better
Gemini performs better on this dimension, supported by observable evidence.

Tie
Both models perform comparably on this dimension, whether both perform well or both perform poorly. Explain which applies.

Insufficient evidence
Available transcripts or artifacts do not support a reliable comparison. Identify the missing evidence; do not infer visual or interaction success from unsupported claims.

Not applicable
This dimension is irrelevant to the UI task prompt. Explain why; missing evidence alone does not make a dimension inapplicable.

Dimensions

1. Understanding the UI request and screenshot. Does the model correctly identify the visible defect, affected component, and requested outcome? Judge against the prompt and reference image; do not reward an unsolicited redesign.

2. Layout, alignment, and spacing. Does the result fix the requested positioning, sizing, alignment, padding, margins, or grid/flex behavior? Check for unintended gaps, overlaps, clipping, and overflow.

3. Typography and visual hierarchy. Are font family, size, weight, line height, wrapping, and text emphasis consistent with the prompt or reference and the existing UI? Check that headings, labels, and body text remain readable.

4. Colors and component styling. Do colors, backgrounds, borders, radii, shadows, and control styles match the requested appearance and surrounding design system? Check consistency across affected components.

5. Images, icons, and visual assets. Are the required images and icons present, correctly sized, aligned, and cropped, with appropriate aspect ratios? Check for missing, distorted, or incorrectly substituted assets.

6. Responsive and adaptive behavior. Does the fix behave correctly at the viewport shown in the prompt and other relevant screen sizes? Check reflow, navigation, wrapping, scrolling, and touch-target sizing without introducing breakpoint-specific defects.

7. UI interactions and state behavior. Do affected buttons, links, forms, menus, tabs, or dialogs work as requested? Check relevant hover, focus, selected, disabled, loading, empty, error, and success states; do not infer working behavior from a static screenshot.

8. Accessibility and usability. Does the affected UI remain usable with keyboard navigation, visible focus, meaningful labels, readable contrast, and appropriate semantic controls? Assess applicable accessibility behavior alongside the requested visual change.

9. UI fix completeness and regression avoidance. Does the patch resolve all UI requirements in the prompt while preserving unaffected components, content, and user flows? Check that it fixes the cause rather than hiding the symptom or hard-coding a single screenshot.

10. Rendered verification and visual evidence. Does the transcript show the updated UI actually rendered and checked against the request? Look for before/after screenshots at comparable viewport and state, interaction checks where relevant, and accurate disclosure of untested behavior. A claimed fix or code diff alone does not prove visual success.

Required justification for every choice

After selecting an answer for each dimension, complete the required Text sub-classification labeled Justification that appears beneath that choice. Explain the preference by citing the relevant prompt requirement and concrete evidence from both transcripts, screenshots, code changes, or interaction checks. For Tie, describe comparable strengths or failures; for Insufficient evidence, state what cannot be verified and what is missing; for Not applicable, explain why the criterion is irrelevant. A model name or restatement of the selected option is not a sufficient justification.

Required evaluation record

Complete all ten preference questions and their justifications in Labelbox. Authors summarize the decisive differences in Author Notes; a different reviewer independently evaluates both transcripts and checks each preference and justification, documenting findings and any disagreement in Reviewer Notes. Keep supporting artifacts linked in the submission.

Overall UI comparison

Explain whether Astra has a significant, verifiable advantage based on the UI requirements central to the prompt and the severity of remaining defects. For a layout-fix prompt, focus on whether the layout is corrected; for a broken-menu prompt, focus on whether the menu works in the relevant states. Explain critical failures and any uncertainty. Preferences must reflect the evidence without assuming a winner. Winning more dimensions alone does not establish a significant advantage.

Folder Structure

Please create one folder with the name **RL Multimodal_[Your Task Name]**. Inside that folder create two folders, **Gemini_[Your Task Name]** and **Astra_[Your Task Name]**, and place the artifacts inside these folders respectively.

```
RL Multimodal_[Your Task Name]/
  Gemini_[Your Task Name]/
    (OpenCode transcript and artifacts)
  Astra_[Your Task Name]/
    (Codex session and artifacts)
```

API Credits and Task Rate

You must use and pay for your own API credits for both Astra and Gemini 3.8 Flash when completing this task. The task rate is substantially higher because you are responsible for these API costs, in addition to the work of creating, validating, and documenting the comparison. Make sure you have sufficient credits for both models before starting.

Setup

Codex + Astra

1. Install Codex:

   ```
   npm install -g @openai/codex
   ```

2. Run Codex from your project directory and configure Astra as instructed.
3. Sessions are saved under `~/.codex/sessions/`.
4. Open them with:

   ```
   open ~/.codex/sessions/
   ```

Then grab the relevant session file.

OpenCode + Gemini 3.8 Flash

1. Install OpenCode:

   ```
   curl -fsSL https://opencode.ai/install | bash
   ```

2. Run `opencode`.
3. Inside OpenCode, run `/connect` → select Google → add your Gemini API key.
4. Run `/models` and select Gemini 3.8 Flash (`gemini-3.8-flash`).
5. To export a transcript:

   ```
   opencode session list
   opencode export <SESSION_ID> > transcript.json
   ```

You can also run `opencode export` and select the session interactively.

Project Upload Instructions

1. Upload the folder or zip file using the attachment button.
2. Enter the information required for submission.
3. Submit the data row.